ParcelTrack Lite Week 5 Report

STTCK3033 | Checkpoint presentation on 22 October 2026

Team: Naszrul, Zikri and Syauqi. Final project submission: Week 14. The calendar date for Week 14 remains to be confirmed.

ParcelTrack Lite supports the RapidExpress case study through Seller registration of parcels, Admin assignment, Rider delivery updates and public status tracking. At Week 5, we have a coherent implementation plan, an initial ERD, a shared repository and runnable frontend interfaces for all three modules. The prototype uses synthetic browser data. Real backend services, database security and later cloud deployment remain planned.

1 Module allocation

Owner	Module	Responsibility
Naszrul	M1	Login and User Management
Zikri	M2	Parcel Registration and Public Tracking
Syauqi	M3	Delivery Update and Admin Dashboard

Database contracts, migration coordination and integration are shared. Each member owns their module implementation, tests and explanation. The separate lecturer allocation sheet requires full names and matric numbers.

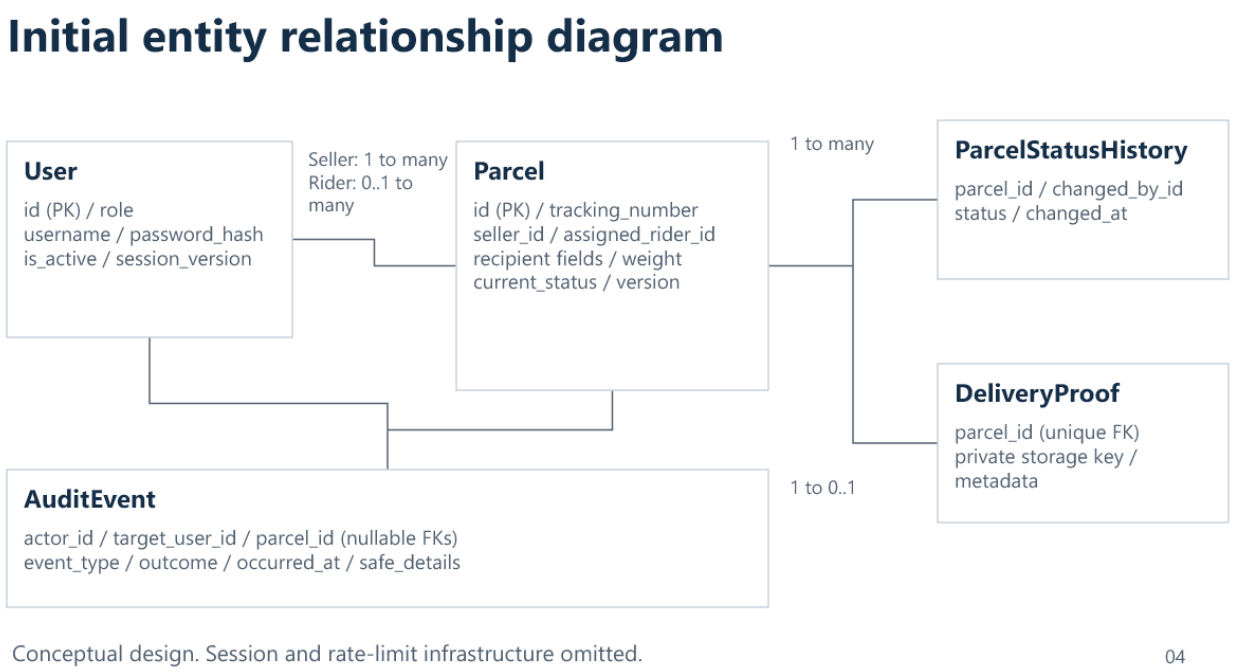
Selected development stack

Layer	Selection
Application	One full-stack Next.js App Router / TypeScript application
Server tools	Shared Zod schemas, pg, node-pg-migrate and Sharp
Data and security	Full local Supabase Docker: PostgreSQL/RLS and private Storage; Argon2id and custom sessions planned
Environment	Local first; cloud hosting deferred until local acceptance
Version control	GitHub repository with module plans and shared contracts

Next.js/PostgreSQL are deliberate team alternatives to the combinations named in the source brief. Lecturer acceptance is not recorded. The target uses the full local Supabase Docker stack. Cloud hosting and a live URL remain deferred; the existing React demo is not yet migrated.

2 Initial ERD and repository

The conceptual model contains five business entities. Seller, Rider and Admin are roles in User. Recipients do not require accounts. Session, rate-limit and maintenance infrastructure are outside this business-entity diagram.



Initial design. The editable diagram is also included on slide 4 of the presentation.

Relationship	Meaning
User to Parcel	One Seller owns many parcels. A parcel has zero or one current Rider. Role constraints require validation.
Parcel to StatusHistory	One or more records per parcel, including Registered at creation. Each records status, actor and time.
Parcel to DeliveryProof	Zero or one proof record; metadata refers to a private Supabase Storage object. Final outcomes require proof under the proposed design.
AuditEvent references	Optional actor, target user and parcel links support account and parcel event records.

Shared data rules

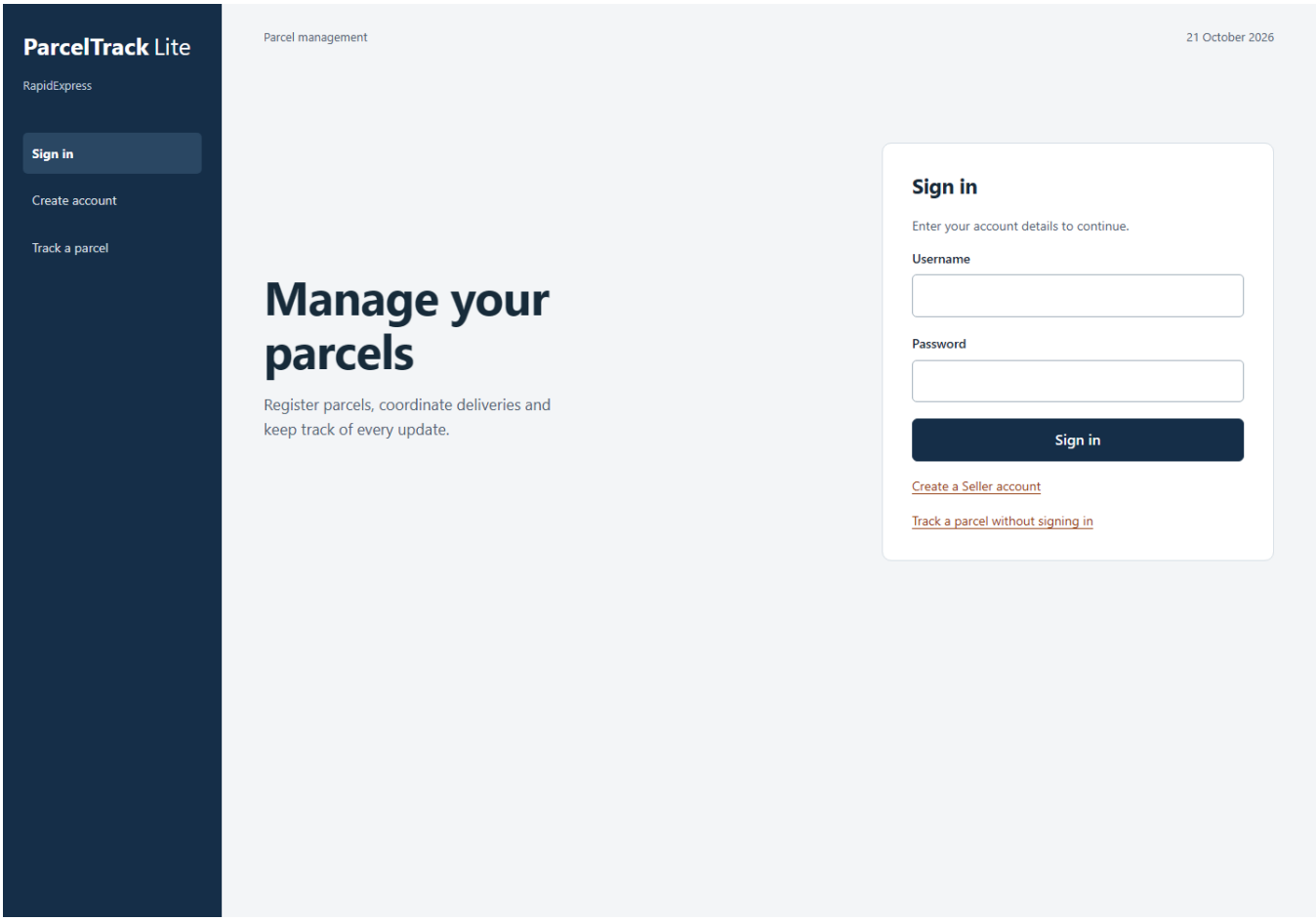
The proposed lifecycle is Registered, Assigned, Out for Delivery, then Delivered or Failed. Admin performs assignment. Only the assigned active Rider performs delivery updates. Terminal outcomes are immutable. The final backend will update status and history atomically and restrict public lookup to status and timestamps.

Repository: https://github.com/Naszrul-NZL/ParcelTrack-Lite

Planning baseline reviewed: commit 9dd50f9. Detailed plans live in docs/modules/ and shared contracts in docs/shared/. The historical React checkpoint prototype is in checkpoint-demo/. The revised local Next.js/Supabase baseline is documented in docs/nextjs-migration.md.

3 M1 Login and User Management

Owner: Naszrul. The interface uses ordinary username/password sign-in, role landing pages, Seller registration, Admin Rider management and a password-change form. Presenter labels and credentials are kept in the guide rather than on product screens.



M1 sample sign-in interface, captured from the running prototype.

Demonstration behaviour

Enter a test username such as seller.a, rider.a or admin, using the presenter credentials. A wrong value displays a generic failure message. Sign-out clears the active identity. Admin can create, enable and disable Riders. Seller self-registration creates a Seller account only.

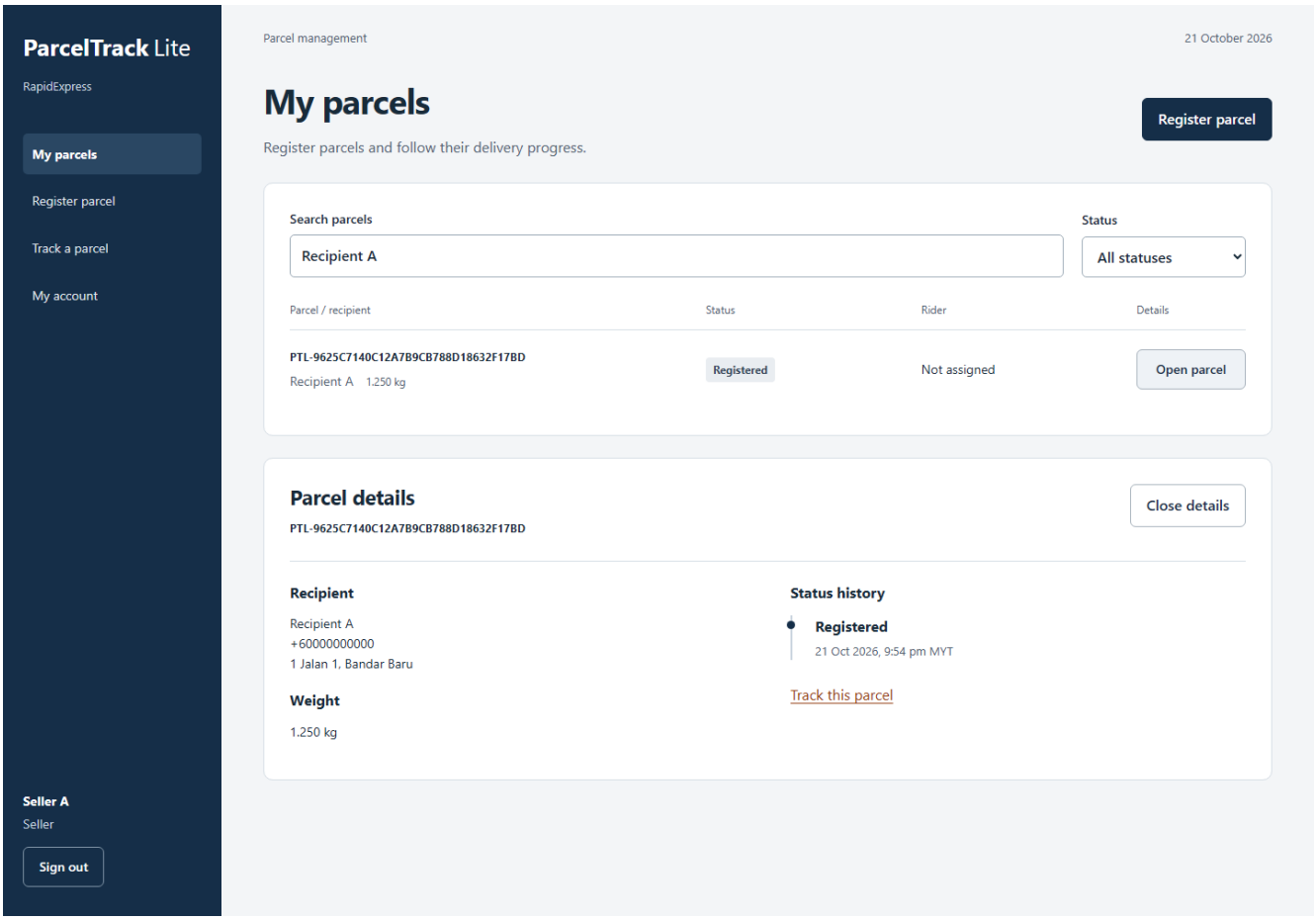
Implementation boundary

Authentication is simulated in the browser. Newly entered account passwords and password changes exist only in the current page memory and reset on refresh; no real password should be used. No password is persisted to localStorage. The final Next.js application must implement Argon2id hashes, verified server-side sessions, CSRF protection, password change and session revocation, account disablement and safe audit logging.

This checkpoint demonstrates the interface and interaction states. Client-side navigation and filters do not prove access control.

4 M2 Parcel Registration and Tracking

Owner: Zikri. The interface supports recipient name, phone, address and weight, a Seller parcel list, search, detail and public status tracking.



M2 newly created synthetic parcel, shown in the Seller list and detail view.

Demonstration behaviour

An empty form produces field errors. A valid fictional parcel receives a browser-generated random tracking token and starts Registered. The same parcel can be searched and opened. Browser storage retains sample parcels after refresh, while refresh clears the demo sign-in.

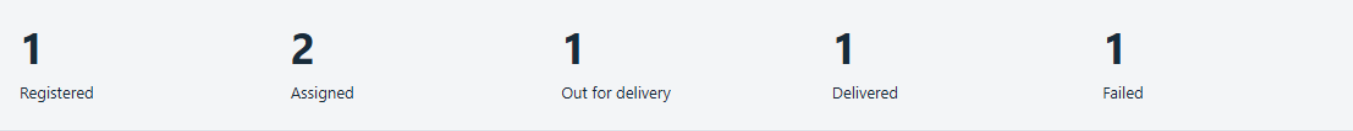
Public tracking accepts the complete tracking token without sign-in. The visible result contains only status and timestamps. The prototype uses a separate public-history projection for this view.

Implementation boundary

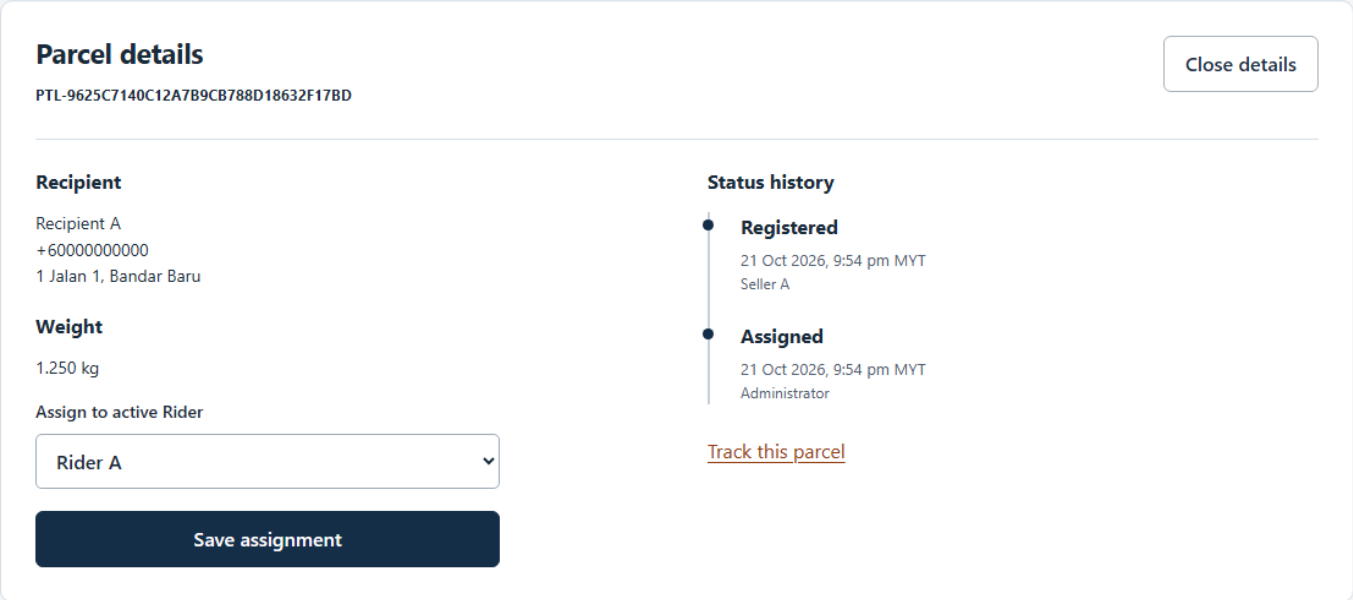
The complete synthetic data set remains in localStorage. Public UI field selection is not a server privacy guarantee. Production work includes authenticated APIs, server validation, parameterized queries, unique tracking constraints, Seller ownership enforced with RLS, restricted tracking lookup, rate limits and retention automation.

5 M3 Delivery Update and Dashboard

Owner: Syauqi. Admin assignment, Rider delivery actions and five status counts share the same prototype parcel data.



Five status counts from the Admin interface.



M3 Admin assigns the newly created parcel to Sample Rider A. The detail and dashboard use the same browser state.

Demonstration behaviour

Admin opens an unfinished parcel and assigns an active Rider. That Rider starts delivery and records Delivered or Failed with one readable sample JPG/PNG. The prototype checks image type, size and decode, and keeps the sample image in browser storage. Status history and dashboard counts update together in the browser.

Implementation boundary

The final backend must enforce the assigned Rider, transition rules and concurrency; validate and sanitize uploaded images; keep private proof storage; and commit history and final status consistently. Existing final sample records are visual fixtures without photos. New prototype finalizations require an image.

6 Security requirements

The following requirements apply to the final implementation. Browser demonstrations are interaction examples, not evidence that these server controls are enforced.

Actor	Allowed access and operation
Seller	Own parcels only; create, list, search and view own records
Rider	Assigned parcels only; authorized delivery transitions and proof
Admin	All parcel overview, assignment, user management and dashboard; no Rider status updates
Public	Tracking by token; status and timestamps only, no recipient data or proof

Policy	Required behaviour
P1	Individual accounts, least privilege and same-day disablement of departing Riders, including existing sessions
P2	At least 10 characters, app-specific passwords and prompt compromise reporting; hashed storage
P3	Recipient data for delivery only; no unauthorized sharing; delete or anonymize 12 months after delivery, including identifying proof
P4	Secrets outside source, synthetic test data, generic user errors and protected technical logs
P5	Record login success/failure and status changes without passwords/full addresses; report incidents within one hour and lock affected accounts

Planned verification

Unit tests cover validation and rules. Integration tests use real PostgreSQL and restricted runtime roles. Browser journeys prove the connected workflow. Negative tests cover wrong roles, other users with the same role, SQL injection, XSS, guessed tokens, rapid lookups and hostile uploads. Cloud checks cover HTTPS, cookies, private ports, persistence and recovery.

The 12-month period above is the case-study policy. This report does not claim legal compliance. RLS, hashing, sessions, CSRF, safe SQL and cloud controls remain unimplemented in this prototype.

7 Progress challenges and next steps

Area	Checkpoint status
Planning	Allocation, stack, ERD, shared contracts, six phases per module, test plans and handoffs prepared
Interfaces	Runnable React prototype across M1, M2 and M3, with one shared sample parcel journey
Prototype verification	Build passes; 6 domain tests pass; browser smoke journey passes 19 checkpoints with no runtime errors
Backend and database	Next.js, actual migrations, sessions, RLS and real private file storage are planned
Final verification and cloud	Full integration/security suite and later cloud deployment have not run
External items	Full allocation details, stack acceptance, later cloud configuration and branch-protection application need evidence

Current challenges and response

Shared users, parcels and statuses can diverge during parallel development. The team will use one system contract and ordered migrations. Browser simulations must be replaced by real APIs without weakening permissions. Each member will review and test agent-assisted implementation and be able to explain their module. Local Supabase setup and the selected stack variation should be settled early.

Proposed teaching-week schedule

Week	Milestone
6	PH1: Next.js and full local Supabase foundation, migrations and test harness
7	PH2: real authenticated module slices on one local stack
8-9	PH3: complete local functions, retire substitutes and verify the production build
10-11	PH4: security regression, failure cases and local recovery
12	PH5: later cloud deployment, scheduling and operational checks
13-14	PH6: final evidence, rehearsal and Week 14 submission

This week mapping is a proposed schedule, not a completion claim. The final calendar date and milestone progress must be confirmed by the team.

8 Demonstration and submission guide

Extract ParcelTrack-Lite-Demo.zip. With Node.js 22.14 or newer installed, double-click start-demo.cmd and open its printed local address. The compiled React files are included, so dependency downloads are not required to run the extracted demo. Keep the server window open.

Before rehearsal, press Alt+Shift+R and confirm Restore data. Initial test usernames are seller.a, seller.b, rider.a, rider.b and admin, all using the fake password DemoOnly10!. Credentials are not displayed on product screens. Use fictional recipient data, the included sample-proof.png and one browser origin for the sequence.

Presenter	Action	Visible outcome
Naszrul	Sign in as seller.a; optionally show one wrong password first	Generic failure then Seller landing page
Zikri	Submit empty form, then create a fictional parcel and copy its tracking token	Field validation, Registered row and initial history
Syauqi	Sign in as admin, search token and assign Rider A	Assigned status and updated count
Syauqi	Sign in as rider.a, start delivery, then Delivered with sample proof	Out for Delivery, then Delivered with history
Zikri	Sign out, open Track a parcel and paste the token	The same parcel history, displaying only status/time

Suggested opening: This is our Week 5 frontend prototype using synthetic browser data. Backend integration, database security and later cloud deployment are planned for the Week 14 project.

If the live browser cannot run, use the presentation screenshots as a walkthrough and state the limitation. The separate demo-guide.md includes recovery steps, optional examples and likely questions.

Source materials

ParcelTrack Lite Project Briefing.pptx, slides 6 to 13. ParcelTrack_Lite_Case_Study.docx, sections 4 to 11. Rubric_Project_STTCK3033_ParcelTrack_Lite (1).docx, sections 2 and 3. Repository documents: docs/initial-erd.md, docs/shared/system-contract.md, docs/shared/phase-framework.md and the three module folders. Team-supplied Week 5/Week 14 timeline and module allocation.

Interface screenshots and prototype results come from the local checkpoint demo. The existing planning documents describe real backend acceptance gates; those gates remain planned.